

```

/**
 * server/services/agent-events.ts – PATCH
 *
 * Fix #5 – Live Gate Cooldown Doesn't Survive Reboots
 *
 * PROBLEM: The 4-hour cooldown for o0vbZwsaeKI (and any future live stream
 * that triggers false positives) is stored in memory. On every reboot, it
 * resets and the system runs 5 HLS checks again from scratch, enqueueing
 * failed Discord announce jobs and burning connection attempts.
 *
 * The fix in April already works correctly (RecordingGate: Both HLS attempts
 * failed → 4h cooldown set) – it just needs to persist that cooldown in the
 * DB so it survives reboots.
 *
 * BEFORE (current in-memory pattern):
 * _____
 * let liveGateCooldownUntil: Date | null = null;
 *
 * function isLiveGateSuppressed(): boolean {
 *   return liveGateCooldownUntil !== null && liveGateCooldownUntil > new Date();
 * }
 *
 * function setLiveGateCooldown(durationMs: number): void {
 *   liveGateCooldownUntil = new Date(Date.now() + durationMs);
 *   log.warn(`[RecordingGate] Cooldown set until ${liveGateCooldownUntil.toISOString()}`);
 * }
 * _____
 *
 * AFTER (DB-persisted pattern):
 */

import { storage } from '../storage.js';
import { createLogger } from '../lib/logger.js';

const log = createLogger('agent-events');

const COOLDOWN_KEY = 'live_gate_cooldown_until';

/**
 * Check if the live gate is suppressed.
 * Reads from DB so the cooldown survives reboots.
 */
async function isLiveGateSuppressed(videoId: string): Promise<boolean> {
  try {
    const key = `${COOLDOWN_KEY}:${videoId}`;
    const raw = await storage.getSystemSetting(key);
    if (!raw) return false;
  }

```

```

const cooldownUntil = new Date(raw);
if (cooldownUntil > new Date()) {
  log.debug(
    `[RecordingGate] ${videoId} cooldown active until ${cooldownUntil.toISOString()} -
suppressing`
  );
  return true;
}

// Cooldown expired - clean up
await storage.deleteSystemSetting(key);
return false;
} catch {
  return false; // fail open - let it check if DB is unavailable
}
}

/**
 * Set a live gate cooldown that persists across reboots.
 */
async function setLiveGateCooldown(videoId: string, durationMs: number): Promise<void> {
  const cooldownUntil = new Date(Date.now() + durationMs);
  const key = `${COOLDOWN_KEY}:${videoId}`;

  try {
    await storage.setSystemSetting(key, cooldownUntil.toISOString());
    log.warn(
      `[RecordingGate] ${videoId} - Both HLS attempts failed. ` +
      `Cooldown set until ${cooldownUntil.toISOString()} (persisted to DB - survives reboot)`
    );
  } catch (err) {
    log.error(`[RecordingGate] Failed to persist cooldown for ${videoId}:`, err);
    // Fall back to in-memory only if DB write fails
  }
}

// — Usage - replace the existing live gate check in agent-events.ts —————
//
// BEFORE:
//   if (isLiveGateSuppressed()) {
//     // skip
//   }
//
// AFTER:
//   if (await isLiveGateSuppressed(videoId)) {
//     // skip
//   }
//
// BEFORE (on double-failure):
//   setLiveGateCooldown(4 * 60 * 60 * 1000);

```

```
//
// AFTER:
//   await setLiveGateCooldown(videoId, 4 * 60 * 60 * 1000);
//
// ——— Storage methods to add to server/storage.ts —————
//
//   getSystemSetting(key: string): Promise<string | null>
//   setSystemSetting(key: string, value: string): Promise<void>
//   deleteSystemSetting(key: string): Promise<void>
//
// Back these with a simple system_settings table:
//   CREATE TABLE IF NOT EXISTS system_settings (
//     key          TEXT PRIMARY KEY,
//     value        TEXT NOT NULL,
//     updated_at   TIMESTAMPTZ DEFAULT NOW()
//   );

export { isLiveGateSuppressed, setLiveGateCooldown };
```